

PROJECT ARCHITECTURE REPORT

# Telecom Customer Churn Predictor

End-to-End Machine Learning & Production Deployment

**Saleem**

Data Scientist & AI Developer

# 1. Executive Summary

---

As a Data Scientist, I often say that building a highly accurate machine learning model in a Jupyter Notebook is only 20% of the job. The real engineering challenge lies in taking that raw mathematical logic and deploying it into a live, scalable, and resilient production environment.

This report details the architecture and engineering decisions behind the **Telecom Customer Churn Predictor**. The objective of this project was to move beyond static data analysis and build a full-stack MLOps pipeline capable of predicting whether a telecommunications customer is at risk of leaving the company based on their usage metrics.

Python

Flask

Scikit-Learn

Pandas

Next.js

Tailwind CSS

Vercel

Render

## 2. Dataset & Model Performance

---

A production deployment is only as valuable as the math powering it. Ensuring robust data provenance and appropriate model evaluation metrics was critical before exposing the API to the front-end.

### The Data Profile

The model was trained on a historical Telecom Customer Churn dataset consisting of over 3,300 unique customer records. The feature space included highly predictive behavioral metrics such as total day/evening/night minutes, international plan subscriptions, account length, and the frequency of customer service calls.

### Champion Model Selection

While complex ensemble methods (like Random Forest) were tested during local model exploration, a **Logistic Regression** pipeline was selected as the champion model for the production environment. This decision was strategic:

- **Interpretability:** Logistic regression provides clear coefficients, allowing business stakeholders to understand exactly which features (e.g., a high volume of customer service calls) are driving the churn risk.

- **Imbalanced Classification Strategy:** In a production telecom environment, churn datasets are highly imbalanced (most customers stay). The model was optimized specifically for **Recall** and **ROC-AUC** to ensure the business effectively captures true churn risks without overwhelming the retention team with false positives.

## 3. System Architecture

---

To ensure scalability and separation of concerns, I decoupled the application into a modern front-end interface and a dedicated containerized backend API.

### Front-End: Next.js on Vercel

The user interface is built using **Next.js and React**, styled with Tailwind CSS, and hosted on Vercel's global edge network. This allows for lightning-fast UI rendering. The front-end manages complex state asynchronously, capturing customer metrics and packaging them into strictly typed JSON payloads.

### Back-End: Python API on Render

The prediction engine lives on a Linux-based **Render** web service running a Python/Flask API. When the frontend sends a POST request, the API receives the data, cleans it, processes it through the pre-trained Scikit-Learn pipeline, and returns a calculated churn probability percentage.

## 4. Engineering Challenges & Solutions

---

Deploying Machine Learning models to the cloud frequently exposes friction points between local training environments and production servers. Overcoming these hurdles was a primary focus of this project.

### CHALLENGE A: CROSS-OS SERIALIZATION (THE PICKLE BUG)

**The Problem:** Scikit-Learn utilizes low-level C-arrays to optimize tree-based algorithms. When models are frozen (.pkl) in a local Windows environment and deployed to a Linux-based cloud server, the OS architecture mismatch often causes severe memory deserialization crashes.

**The Solution:** By architecting the pivot to a math-based Logistic Regression model, I bypassed the Windows-to-Linux C-array corruption. Because Logistic Regression relies on coefficient mathematics, it serializes perfectly across different operating systems, ensuring a stable, zero-downtime deployment.

### CHALLENGE B: DYNAMIC FEATURE PADDING

**The Problem:** Scikit-Learn pipelines are exceptionally rigid regarding data shapes. The model expects over 20 columns, but for UI simplicity, the front-end dashboard only passes the 5 most critical metrics. Passing a 5-column dataframe to a 20-column model results in an immediate shape-mismatch crash.

**The Solution:** I engineered a dynamic Python middleware layer in the Flask API. Upon receiving a request, the API queries the loaded model's memory to extract the exact list of feature names it expects. It generates a temporary Pandas DataFrame filled with safe, zero-padded baseline values, and maps the 5 live data points from the React front-end into their precise column locations.

## 5. Conclusion & Future Roadmap

The Telecom Customer Churn Predictor successfully bridges the gap between Data Science and Software Engineering. It proves that predictive analytics can be cleanly integrated into modern web architecture to provide immediate, actionable business intelligence.

**Future Iterations:** As the system scales, the next architectural evolution will involve implementing a native Linux training script (CI/CD) on the Render server. By commanding the server to train and

serialize the model natively upon deployment, we will bypass all cross-OS pickling limitations, allowing for the seamless deployment of heavier algorithms like XGBoost.